

Future Control Moves Memory: A Long-Horizon Moved-Bottleneck Diagnostic for Synthetic Agents

Jawaun Brown 2026-07-03

Abstract

This paper asks a narrow question about finite neural agents: when one early state variable becomes future-critical for a delayed decision, does the agent's memory geometry become specifically sensitive to that variable, and does that sensitivity move when the critical variable moves?

We introduce the **long-horizon moved-bottleneck diagnostic**. Four early clue slots are matched for salience and frequency. One slot is selected as the future-critical bottleneck, moved across registered positions, and queried only after a long delay. The primary metric is not just final accuracy; it is the relative displacement of the final hidden state under single-slot bit flips. The registered gate requires the final-state sensitivity peak to follow the moved critical slot while remaining absent in a visible-control condition.

The result is positive across a ladder of increasingly agent-like synthetic regimes. A transformer on Modal L4 solves the base delayed-memory task, survives longer horizons through sequence length 384, and preserves the moved bottleneck through closed-loop external-state handoff, repair after tool error, parsed structured calls, multi-field tool schemas, stochastic API failures, an 8-slot larger argument namespace, and finally an alias-rich argument surface with three equivalent aliases per canonical slot. Across the confirmed Modal reports, the relevant bottleneck groups reach 1.000 closed-loop accuracy and positive moved-slot specificity, while visible controls keep specificity near zero.

The allowed claim is deliberately modest: in this synthetic neural-agent setting, **future control relevance can move finite memory-state and tool-commitment sensitivity**. This is not a production-agent benchmark, language-agent benchmark, or consciousness claim. Its value is as a cheap, reproducible diagnostic for whether an agent's internal state tracks the variables that will later control action.

1. Motivation

Many long-horizon agent evaluations ask whether an agent eventually gives the right answer. That is necessary but weak. A system can succeed by a shortcut, by storing everything uniformly, by using local query cues, or by relying on a privileged final token. The moved-bottleneck diagnostic tests a sharper property: whether the agent's internal memory metric follows the variable that will matter later.

This matters for three fields.

1. **Interpretability**: it gives a controlled way to ask where future-relevant information lives in a model's hidden state.
2. **Agent evaluation**: it tests delayed commitment and repair, not just answer selection at the final step.
3. **Safety and monitoring**: it suggests a cheap diagnostic for variables that silently become control bottlenecks in long-horizon systems.

The experiment is intentionally synthetic. That is a feature, not a defect, for this stage: the task isolates memory transport, critical-variable movement, and tool-interface hardening before introducing the confounds of natural language and real APIs.

2. Diagnostic Design

Each episode contains matched early clues:

```
slot_0: bit b0
slot_1: bit b1
```

```

slot_2: bit b2
slot_3: bit b3
...
delayed query

```

In the `bottleneck` condition, the correct final action is the bit at one registered critical slot. The critical slot is moved across runs. In the `visible_control` condition, the final answer is visible at the query token and the early clues are matched distractors.

For each trained cell, the metric intervention flips one early clue bit at a time and measures the final hidden-state displacement. Slot densities are z-scored within model. The primary specificity statistic is:

```

z_density(critical_slot) - mean z_density(noncritical_slots)

```

The core gates are:

- behavior: bottleneck accuracy at least 0.90;
- metric transport: bottleneck memory-specificity CI lower bound above zero;
- rank: the critical slot ranks above chance among registered slots;
- visible-control null: visible-control specificity remains below 0.50.

The tool-interface regimes add gates for committed slot/value accuracy, schema validity, parsed action validity, repair behavior, stochastic failure handling, and no-op behavior after successful first calls.

3. Evidence Ladder

The experiment was advanced through a sequence of regimes. Each regime keeps the moved critical slot and visible-control null, then changes one surface of the agent's interaction with future-relevant information.

Regime	What changed	Cells	Key signal	Report key
Base moved bottleneck	Delayed query over four matched clue slots	64	Bottleneck final accuracy 1.000; memory specificity +2.309; visible null	base
Horizon stress	Delay lengths 128, 256, 384	96	All lengths preserve final accuracy 1.000 and moved-slot specificity	horizon
Closed-loop tool handoff	Model commits slot/value to recover external state	32	Tool bottleneck final accuracy 1.000; slot/value commitment 1.000	commit
Repair bottleneck	First tool attempt receives an error; model must repair	48	Repair condition final accuracy 1.000; repair slot/value 1.000	repair
Structured tool call	One parsed JSON-like action token replaces direct heads	48	Direct and repair structured calls pass schema and parsed-slot gates	structured
Multifield schema	Separate opcode, slot, and value fields	48	Multifield direct/repair groups pass field, schema, and parsed-value gates	multifield
Stochastic tool failure	First-call success/failure sampled per episode	32	Failed repair and success no-op both 1.000; failure rate 0.506	stochastic
8-slot stochastic	Larger argument namespace and longer sequence	64	8-slot stochastic gates pass; memory specificity +3.023	8slot
Alias argument surface	Three equivalent aliases per canonical slot	32	Alias parsed slot/value, failed repair, and success no-op all 1.000	alias

All confirmed sweeps used Modal L4, not H100/H200. The timeout-based conservative spend guard for the listed

confirmed reports is under \$130 in aggregate; individual recent passes used guards of 8.63 to 17.26. Actual runtime was much lower than the timeout budget, with the latest alias pass averaging 15.07 seconds per remote cell.

Report keys map to committed summaries under `experiments/long_horizon_bottleneck/results/`: `base = modal_transformer_l4_8seed_2026_07_02.md`, `horizon = modal_transformer_l4_horizon_4seed_2026_07_02.md`, `commit = z_closed_loop_tool_commitment_l4_4seed_2026_07_02.md`, `repair = zz_tool_recovery_l4_4seed_2026_07_02.md`, `structured = zzz_structured_tool_call_l4_4seed_2026_07_03.md`, `multifield = zzzz_multifield_tool_schema_l4_4seed_2026_07_03.md`, `stochastic = zzzzz_stochastic_tool_failure_l4_4seed_2026_07_03.md`, `8slot = zzzzzz_stochastic_tool_failure_8slot_l4_4seed_2026_07_03.md`, and `alias = zzzzzzz_alias_argument_surface_l4_4seed_2026_07_03.md`.

4. Latest Alias-Surface Result

The alias pass is the most natural-language-adjacent synthetic regime in the current ladder. It replaces the compact slot id with an alias argument field: four canonical slots, three aliases per slot, plus missing and malformed sentinels, for a vocabulary size of 14. The loss accepts any alias belonging to the correct canonical slot. The evaluator parses aliases back to canonical slots before applying the same stochastic repair/no-op checks.

The Modal L4 run used 32 cells: 2 conditions, 4 moved critical slots, and 4 seeds. Results:

Group	Closed-loop final	First parsed slot/value	Failed repair slot/value	Success repair no-op	Failure rate	Memory specificity	Gate
alias_stochastic_bottleneck/transformer	1.000	1.000 / 1.000	1.000 / 1.000	1.000	0.506	+2.309	pass
alias_visible_control/transformer	1.000	n/a	n/a	n/a	0.506	+0.000	pass

This rules out a small but important alternative: the prior result was not only an artifact of a single compact slot-id classifier. The moved bottleneck survives a synonym-like argument surface under stochastic first-call failure and conditional repair.

5. Interpretation

The core result is a **metric transport** result. The future-critical variable does not merely determine the final label; it becomes the slot to which the agent's final hidden state and commitment heads are specifically sensitive. When the critical slot moves, the sensitivity peak moves with it. When the final answer is visible at query time, the early-slot peak disappears.

The tool regimes add a second claim: the moved bottleneck survives when the agent has to externalize the critical variable through a tool interface. It must commit an argument, receive or fail to receive an external return, repair when needed, no-op when not needed, and still preserve the correct delayed decision.

This makes the diagnostic more relevant to long-horizon agents than to ordinary sequence classifiers. The bottleneck is not just "remember bit 2." It becomes "know which early variable must later be committed through an interface, recover it under feedback, and ignore matched distractors."

6. Boundaries

The strongest honest statement is:

Future control relevance can move finite memory and commitment sensitivity in a synthetic long-horizon neural agent, and that moved bottleneck survives progressively harder parsed tool-interface regimes.

The result does not establish:

- production API reliability;
- autonomous language-agent tool use;
- robustness under natural-language prompts;
- multi-step planning in an open environment;
- human cognition or consciousness.

The latest alias regime is still a fixed classifier surface. It is a bridge toward natural language, not natural language itself.

7. Why This Is Valuable

The experiment's field value is not that it solves a benchmark. It gives a small, inspectable test for a pattern that current agent evaluations often miss: future relevance can reorganize internal memory geometry and action commitment.

A useful follow-on benchmark could ask, for a real agent or language model:

1. Which early facts become future-critical under a delayed objective?
2. Do internal representations or tool-call logits become specifically sensitive to those facts?
3. Does that specificity move when the objective moves?
4. Does it remain absent when the answer is visible later?
5. Does it survive tool failure, repair, and aliasing of argument names?

That is a sharper evaluation target than final-task success alone.

8. Remaining Work

The synthetic mechanism ladder is now complete enough to write up and share. The next regimes are optional and answer different questions:

- **Text-prompt tool use:** replace alias classifiers with real text arguments and parser-facing JSON.
- **LLM-agent transfer:** run a small language model or API model through a prompt-level version of the moved bottleneck.
- **Multi-step planning:** require two or more future commitments where the critical bottleneck changes after intermediate feedback.
- **Interpretability probes:** compare the hidden-state metric to attention, activation patching, or linear probes over the critical slot.

The most valuable immediate next step is not another synthetic GPU sweep. It is to turn this into a compact external artifact: a paper, benchmark card, or agent-evaluation proposal with the Modal reports and code paths linked.

9. Reproducibility

Experiment code lives in `experiments/long_horizon_bottleneck/`.

Primary runner:

```
doppler --scope /Users/jawaun/superoptimizers run -- \
  uvx --python 3.12 --from modal modal run \
  experiments/long_horizon_bottleneck/modal_stochastic_tool_failure_sweep.py \
  --seeds 4 --train-steps 900 --architectures transformer \
  --conditions alias_stochastic_bottleneck,alias_visible_control \
  --critical-slots 0,1,2,3 \
  --aliases-per-slot 3 \
  --failure-probability 0.5 \
  --budget-usd 25 \
  --out artifacts/long_horizon_bottleneck/alias_argument_surface_14.json
```

Local verification for the code paths:

```
uvx ruff check experiments/long_horizon_bottleneck tests/test_long_horizon_bottleneck.py
uvx ty check experiments/long_horizon_bottleneck tests/test_long_horizon_bottleneck.py
python -m compileall -q experiments/long_horizon_bottleneck tests/test_long_horizon_bottleneck.py
python -m pytest tests/test_long_horizon_bottleneck.py -q
```

The raw Modal artifacts are kept under gitignored `artifacts/`; committed result reports in `experiments/long_horizon_bottleneck/results/` summarize every run.